Broken Object Property Level Authorization

Introduction

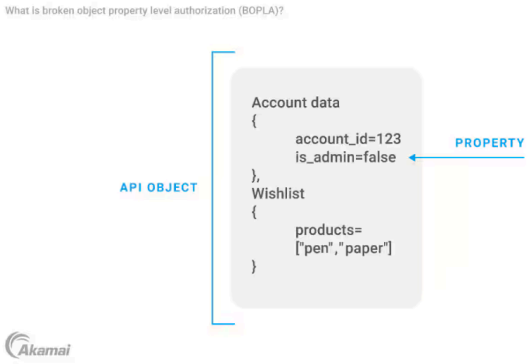
Broken Object Property Level Authorization (BOPLA) is a security vulnerability in application programming interfaces (APIs) that, because of inadequate authorization checks, may allow hackers to view or manipulate the properties of sensitive objects. This may potentially result in data loss, exposure of sensitive data, privilege escalation, or account takeover.

BOPLA is listed at #3 in the 2023 list of the OWASP Top 10 API security issues.

What is BOPLA Vulnerability ?

When websites / applications retrieve data and populate fields on the front end by querying APIs for information about the objects like database records or files, each object typically contains several data points known as “properties”. For example: a user account may include properties like email_address, password, etc.. Most importantly, it may also include **is_admin,** which indicates whether the user has administrative privileges.

During an API call, APIs often send more information than is technically required in the API response, leaving it to the client application to extract the data it needs and render a view for the user. This vulnerability creates risk in an API endpoint in two different ways.

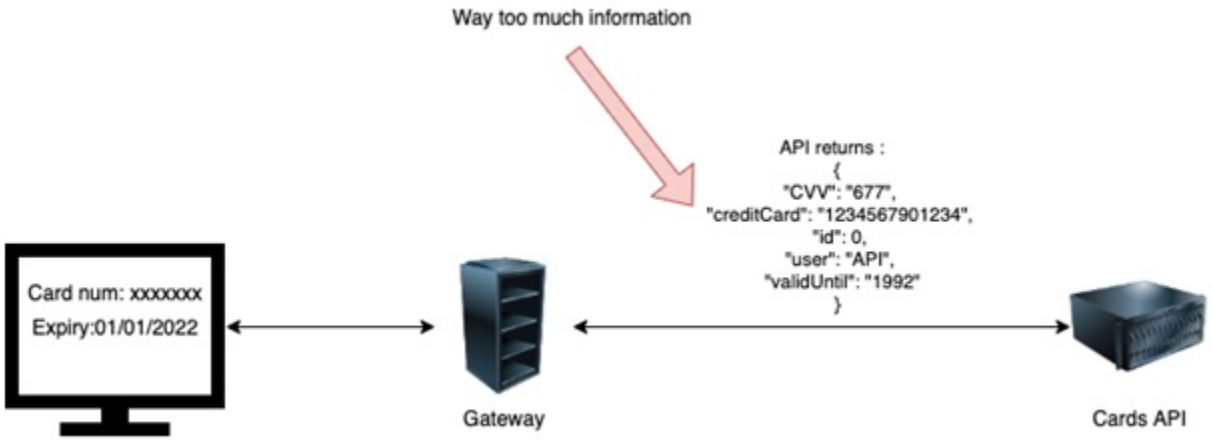


What is the impact of this issue ?

- BOPLA vulnerability allows attackers to gain unauthorized access to sensitive data.
- Attackers may leverage BOPLA vulnerability to escalate their privileges within an application. Also known as **mass assignment.**
- Security breaches resulting from BOPLA vulnerabilities can damage an organization's reputation and erode customer trust.

Excessive Data Exposure

In the context of API and BOPLA, excessive data exposure vulnerability occurs when the API provider responds to a client's request with more data than necessary.

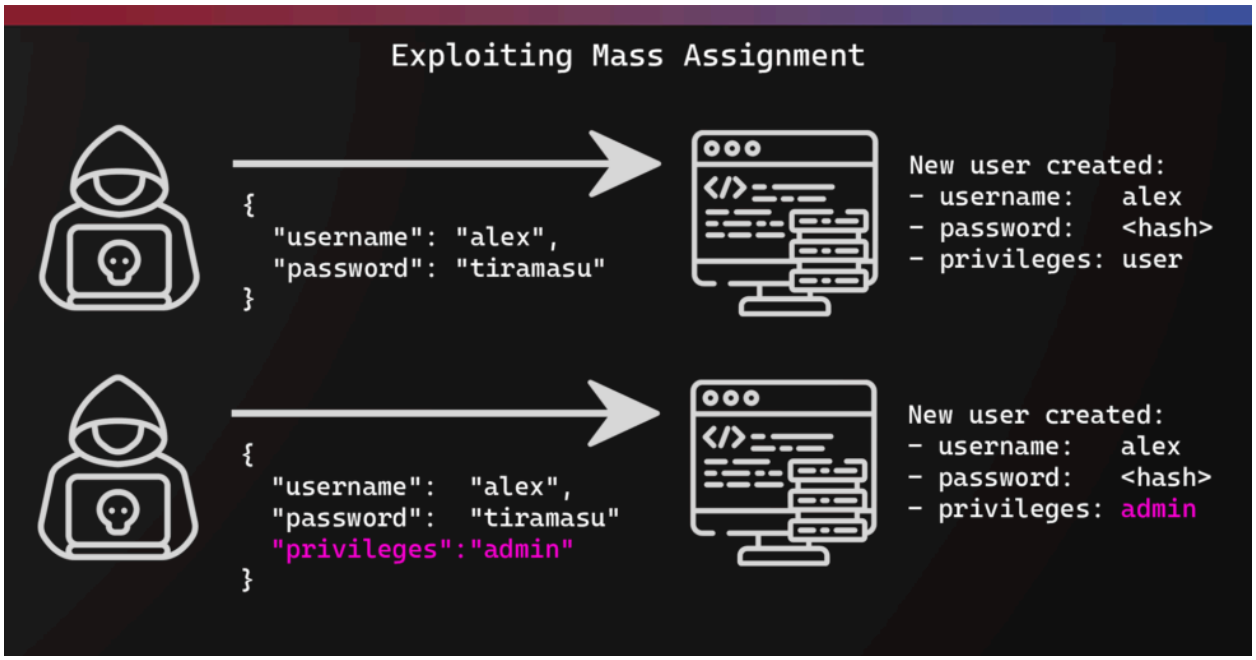


For example,

Excessive data exposure occurs when a user request, such as a simple GET /user?id=233 to a web application, retrieves information that should be restricted and only visible to the currently logged-in user.

Mass Assignment

Mass assignment vulnerability occurs when a user is able to initialize or overwrite server-side variables which are not intended by the application.



For example,

When a user is able to interject the transmission of their data in a web application and then be able to modify it by adding extra properties (such as, “is_admin”: true, “admin”: true, “privileges”: “admin”) that allow them to have admin privileges, then it is known as **Mass Assignment.**

Case Study

Imagine a social media application where users can update their profile information.

Scenario: User Profile Update

A regular user, Alice, wants to update her display name on the application. When she submits the change, the frontend application sends a PUT / GET request to the backend api.

I.e

```
PUT /api/v1/users/self
{
    “displayName”: “AliceNewName”,
    “bio”: “Loves writing reports”
}
```

The Vulnerability (Mass Assignment)

The application’s api is vulnerable to Mass Assignment because it accepts and processes any field passed in the request body, not just the fields intended for user controlled updates.

The user, Alice, is smart. She inspects the API request and adds a hidden property she knows exists on the user object.

```
isAdmin
```

Alice's Malicious Request

```
PUT /api/v1/users/self
{
  "displayName": "AliceNewName",
  "bio": "Loves writing reports",
  "isAdmin": true
}
```

Because the API does not enforce strict authorization to ensure only allowed fields are modified, the server processes the request and silently updates Alice's account, granting her administrative privileges.

The BOPLA vulnerability, in this case is Mass Assignment which allows an attacker to overwrite a critical object property `isAdmin` to perform privilege escalation.

Scenario: Viewing a Public Profile

The same web application also allows users to view other user's public profiles.

When a user, Bob, visits Alic's profile, the frontend application calls an API endpoint:

```
GET /api/users/233
```

The Vulnerability (Excessive Data exposure)

The api is configured to fetch the complete user object from the database including fields that should only be visible to the user themselves or to the administrators. For example, email address, IP address, etc..

The api responds with

```
{
  "id": 233,
  "displayName": "Alice",
  "bio": "Loves writing reports",
  "emailAddress": "alice.private@example.com",
  "lastLoginIP": "192.168.1.1",
  "private_api_key": "sk-1a2b3c4d5e6f7g8h9i0j",
  ...
}
```

The front end application only displays **displayName** and **bio.** However, because the API sent the full object, an attacker like Bob who intercepts the traffic can view other sensitive properties, such as Alice's email, her IP address and maybe even their api key. This is an example of BOPLA resulting from Excessive Data exposure, where the API exposes more object properties than the client application requires or the requesting user is authorized to view.

Conclusion

Broken Object Property Level Authorization (BOPLA) is a critical API security vulnerability that ranks highly on the OWASP Top 10 list. It arises from lack of authorization checks, allowing attackers to view or manipulate sensitive object properties.

BOPLA manifests primarily as **Excessive Data Exposure**, where an API returns more data than the client needs, and **Mass Assignment**, where an attacker can overwrite unauthorized properties like an **`isAdmin`** flag to escalate privileges.

Effective defense requires strictly defining and enforcing authorization at the property level for both reading and writing data in API responses and requests.

References

https://www.akamai.com/glossary/what-is-bopla

https://tcm-sec.com/exploiting-mass-assignment-vulnerabilities/

https://www.wallarm.com/what/excessive-data-exposure